Rajarata University of Sri Lanka

Department of Computing

ICT 1306 — Object Oriented Programming | Group Assignment

Food Hub

Food Delivery Management System

A web application built with Vue 3, Vite, a C++17 REST API (Crow) and SQLite, with the six core Object-Oriented Programming concepts explained through its C++ source code.

Team & Scenario Details

Item	Details
Team name	**[Enter team name]**
Chosen scenario	**Food Hub** — a small food-delivery business: customers browse a menu, fill a cart and place orders priced in LKR; administrators maintain the menu and stock, move orders through a status workflow and manage user accounts.
Course / Assignment	ICT 1306 — Object Oriented Programming, Group Assignment: *Creating a Desktop Application with OOP Concepts*
Project deadline	13 October 2026
Solution / Project	FoodHub_Web_VSCode — `frontend/` (Vue 3 + Vite) and `backend/` (C++17, CMake, Crow, SQLite3), developed in Visual Studio Code

No.	Member name	Index number	Contribution
1	K.G.C. Lakshitha	ICT/2024/076	Customer module
2	R.B.K.D. Gunasekara	ICT/2024/040	Order module
3	K.D.S.D. Gunasiri	ICT/2024/041	Item & Ingredient
4	H.B.M.I.G. Dasanayaka	ICT/2024/024	Rider / Vehicle
5	A.M.N.C. Vijayasingha	ICT/2024/125	Login / Dashboard

Submission Links

Resource	Link
5-minute demo video (YouTube, unlisted)	**https://youtu.be/PPedzbxg-5Q?si=rnJchHZDjowjFEqp**
GitHub repository (public)	**https://github.com/chamilaki2004-ops/Food-delivery-software.git**

Abstract

Food Hub is a web application that digitises the ordering and administration of a small food-delivery business. Customers register, browse a searchable menu, build a cart and place orders; administrators manage food items, images and stock, move orders through a status workflow, enable or disable accounts and watch a summary dashboard. The user interface is a Vue 3 single-page application built with Vite. It talks over REST and JSON to a C++17 back end built on the Crow framework, which stores all data in a SQLite database. Passwords are protected with salted PBKDF2-HMAC-SHA256, sessions use random tokens that are stored only as SHA-256 hashes, and every database statement uses bound parameters. This report documents the scenario, the technologies, the architecture and database design, the user interface, the class model, and one worked example from the real source code for each OOP pillar: class, object, encapsulation, abstraction, inheritance and polymorphism. It also states openly where the project differs from the assignment brief.

1. Introduction and Scenario

1.1 Problem statement

Small food-delivery businesses commonly take orders by phone or message and keep stock and sales in notebooks. This leads to lost orders, wrong totals, items being sold after they run out, and no simple way for customers to see what happened to an order. Staff also need controlled access so that only authorised administrators can change the menu and the order status.

1.2 Proposed solution

Food Hub is a role-based web system. A visitor signs in or creates an account and is routed by role. Customers see **Explore Menu**, **My Cart** and **My Orders**; administrators see **Dashboard**, **Food Items**, **Orders** and **Users**; both see Notifications and Profile. When a customer places an order, the server totals the cart, checks stock, creates the order and its lines, reduces stock, empties the cart and notifies the customer inside one database transaction. Administrators then move the order through Pending, Confirmed, Preparing, Ready, Completed or Cancelled, and the customer is notified at each step.

1.3 Objectives

- Provide secure registration and sign-in: salted password hashing, expiring session tokens and a role check on every protected route.
- Let customers browse, search and filter the menu, manage a cart and place orders with automatic LKR totals.
- Let administrators add, edit and deactivate food items, upload images and keep stock levels correct.
- Keep orders consistent: transactional checkout, stock reduction, a status workflow and notifications.
- Give administrators an overview: customers, food items, orders, completed order value and monthly order counts.
- Explain the C++ code through the six OOP concepts: class, object, encapsulation, abstraction, inheritance and polymorphism.

1.4 Scope

Food Hub is a locally hosted web application. The Vite development server (localhost:5173) proxies `/api` and `/uploads` to the C++ API (localhost:8080), which keeps its data in `backend/data/foodhub.db`. It targets Windows 11 with Visual Studio Code, MSVC, CMake and vcpkg. Online payment, riders and delivery vehicles, e-mail delivery, category management and a detailed audit interface are outside the current scope. Money is stored in LKR minor units (cents) and displayed in LKR.

1.5 Compliance with assignment requirements

Assignment requirement	What Food Hub provides	Status
At least 2 classes demonstrating the 6 OOP concepts	User-defined C++ types `St` (RAII statement wrapper) and `Identity`, plus library classes used by the code (`crow::SimpleApp`, `std::runtime_error`, `std::mutex`). Inheritance and polymorphism are shown through the standard-library exception hierarchy and function overloading; there is no user-defined derived class yet (Section 7).	**Partly met**
GUI with minimum 3 screens (WinForms or MFC recommended)	A browser-based Vue 3 interface with 10 views: sign-in, menu, cart, orders, notifications, profile, dashboard, food items, order management and users. It is not a WinForms/MFC desktop application, so approval from the lecturer is required.	**Differs**
Real database, at least two record types with add / view / update / delete	SQLite through its C API. Food items and cart lines support create, read, update and delete (food delete = deactivate). Users, orders and notifications support the operations listed in Section 4.	**Met**
Report with team and scenario, class list, one example per pillar, class diagram	Cover page, Section 6 (class list and class diagram) and Section 7 (worked examples).	**Met**
5-minute demo video (YouTube, unlisted) and public GitHub URL	Both links are on the cover page.	**Met**

Note: the assignment recommends MySQL; this project uses SQLite, an embedded relational database accessed through its C API, so the same CRUD principles apply. The statuses above follow the project's own notes (`docs/ICT1306_NOTES.md`), which say the web version must not be described as automatically satisfying the desktop brief.

2. Technologies and Tools Used

The list below was taken from the project files (`backend/CMakeLists.txt`, `vcpkg.json`, `frontend/package.json`, `vite.config.js`, `.vscode/extensions.json`) and the source code.

Technology	Version / detail	Purpose in Food Hub
C++17	CMAKE_CXX_STANDARD 17; built with MSVC from Visual Studio	The whole back end: routes, validation, business rules and data access (backend/main.cpp).
Crow	Installed through vcpkg (vcpkg.json lists crow and openssl); Crow 1.3.4 is named in COMPILER_FIX.md	REST routing with CROW_ROUTE, requests and responses, JSON (crow::json), port 8080, concurrency(4).
SQLite3	Amalgamation 3.54.0 compiled into the executable (backend/third_party/sqlite)	Embedded relational database in data/foodhub.db: WAL mode, foreign keys, CHECK constraints.
OpenSSL	find_package(OpenSSL); linked as OpenSSL::Crypto	PBKDF2-HMAC-SHA256 (210,000 iterations), SHA-256 digests, RAND_bytes, CRYPTO_memcmp.
CMake	3.20 or newer	Builds the `foodhub` executable; uses the vcpkg toolchain file.
vcpkg	Manifest file vcpkg.json; VCPKG_ROOT variable	Downloads and builds the C++ dependencies.
Vue 3	^3.5.0, single-file component with <script setup>	Reactive single-page UI (App.vue): state, computed filters, role-based views.
Vite	^6.0.0 with @vitejs/plugin-vue ^5.2.0	Development server on port 5173 and a proxy for /api and /uploads.
Lucide Vue Next	^0.468.0	Icons for navigation, buttons and statistics cards.
Chart.js	^4.4.0	Declared in package.json. The current dashboard bars are drawn with HTML and CSS, so the library is not yet imported.
Browser Web APIs	fetch, sessionStorage	fetch() calls the API with a Bearer token; sessionStorage keeps the token for the browser tab.
Node.js and npm	Node 20.19+ or 22+	Installs the front-end packages and runs Vite.
Visual Studio Code	Extensions: C/C++, CMake Tools, Vue - Official	Editor and build workflow for both parts of the project.
Git / GitHub	Public repository (link on the cover page)	Version control and source submission.

2.1 Why these choices

- **C++17 with Crow** produces one fast executable, and its types (an RAII wrapper and small structs) are good material for explaining OOP.
- **SQLite** needs no separate database server and keeps everything in one file, so the project runs on any machine. Bound-parameter statements prevent SQL injection.
- **OpenSSL** provides audited cryptography, so the project does not invent its own password hashing or random numbers.
- **Vue 3 with Vite** gives a responsive interface with live search, instant feedback and a fast development loop.
- **CMake with vcpkg** keeps the back-end build repeatable: two configure/build commands after setting VCPKG_ROOT.

3. System Architecture

Food Hub follows a layered client-server architecture. The Vue interface sends JSON requests to the C++ API; each route handler checks the caller, validates input and uses small data-access helpers to read or write the SQLite database; results travel back as JSON.

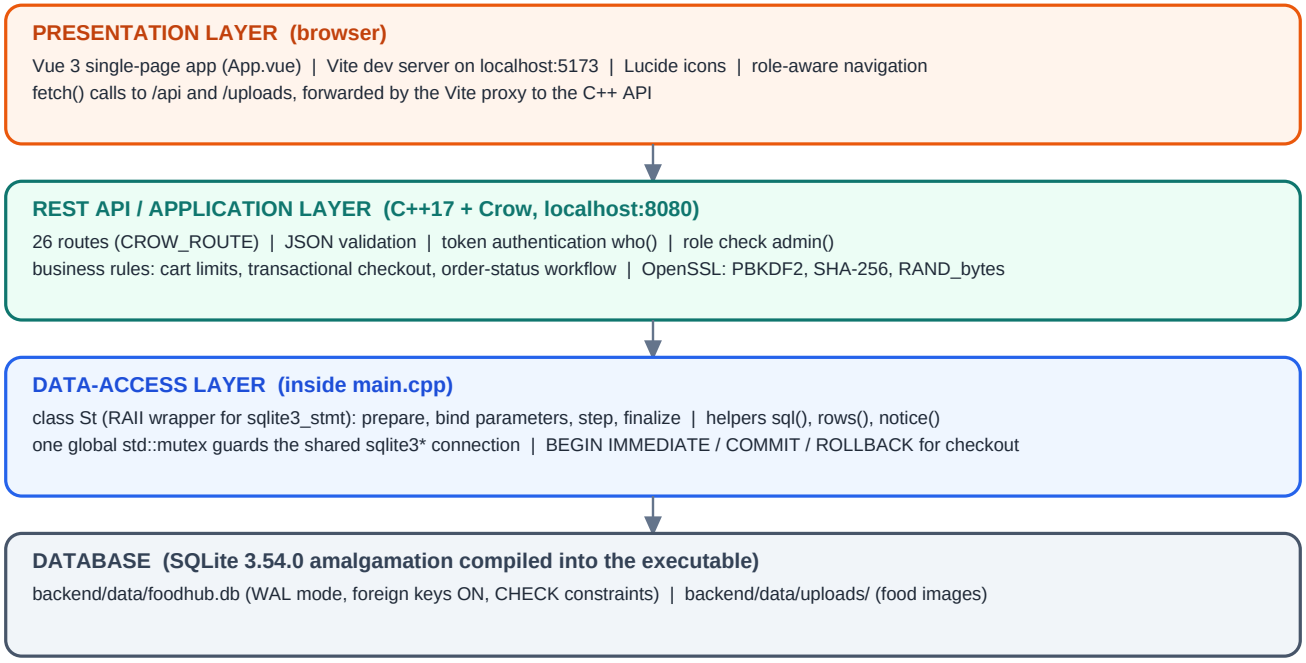


Figure 1 — Layered architecture of Food Hub

3.1 Application workflow

- **Start-up:** `main()` calls `init()`, which creates `data/uploads`, opens `data/foodhub.db`, enables foreign keys and WAL mode, creates nine tables and seeds seven categories (Rice & Curry, Burgers, Pizza, Noodles, Beverages, Desserts, Other). Crow then listens on port 8080 with four worker threads.
- **Registration:** POST /api/auth/register checks name, username (3+ characters), e-mail and password (8+ characters). An administrator account also needs the code held in the FOODHUB_ADMIN_CODE environment variable. The password is salted with 16 random bytes and hashed with PBKDF2.
- **Sign-in:** POST /api/auth/login finds an active user by username or e-mail, compares the hashes in constant time and returns a random 32-byte token. Only the SHA-256 hash of the token is stored in `sessions`, with a 24-hour expiry.
- **Authorisation:** protected routes call `who()`, which joins sessions and users using the Bearer token; admin routes also call `admin()`.
- **Ordering:** customers set cart quantities with PUT /api/cart/<id>; POST /api/orders then runs the checkout transaction.
- **Administration:** admins add, edit, deactivate and illustrate food items, change order status, enable or disable users and read analytics.
- **Front end:** App.vue keeps the token and user in sessionStorage and calls `refresh()` on every navigation to reload the data for the current role.

3.2 Key design decisions

- **Bound parameters everywhere:** every statement is prepared by `St` and values are bound with `b()`; user input is never concatenated into SQL.
- **One transaction for checkout:** BEGIN IMMEDIATE ... COMMIT, with ROLLBACK on any error, so the order, its lines, the stock reduction, the cart clear and the notification succeed or fail together.
- **Money as integers:** prices and totals are stored in minor units (price_minor, total_minor) to avoid floating-point rounding; the UI divides by 100.
- **Soft delete:** deleting a food item sets active = 0, and order lines keep a copy of the food name and unit price, so history stays correct.
- **Serialised data access:** a global `std::mutex` is locked in each handler to protect the single SQLite connection.
- **Safe uploads:** maximum 5 MB, MIME type and file signature checked (PNG, JPEG, WebP), random file name, and /uploads/<name> refuses names containing / or ..

4. Database Design

The database is the SQLite file `backend/data/foodhub.db`. `init()` creates nine tables; the diagram lists their main columns as written in the CREATE TABLE statements.

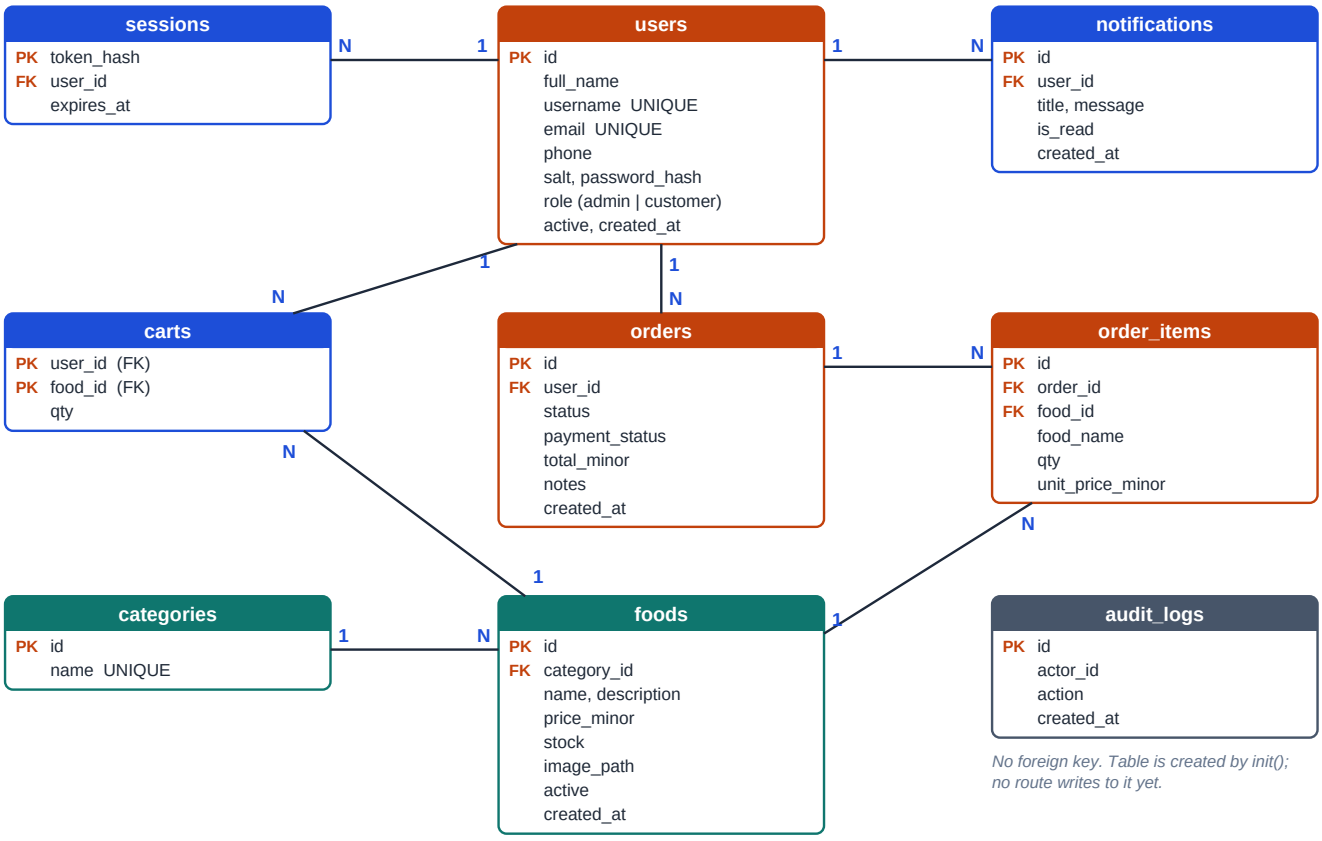


Figure 2 — Entity-relationship view of the FoodHub database (PK = primary key, FK = foreign key)

4.1 Table dictionary and implemented operations

Table	Purpose	Key columns	C	R	U	D
users	Customer and administrator accounts	id, username, email, salt, password_hash, role, active	✓	✓	✓	–
sessions	Sign-in sessions (token hash, expiry)	token_hash, user_id, expires_at	✓	✓	–	✓
categories	Menu categories (seeded by init)	id, name	✓	✓	–	–
foods	Menu items with price, stock and image	id, name, category_id, price_minor, stock, image_path, active	✓	✓	✓	✓
carts	Items a customer has chosen	user_id + food_id (composite key), qty	✓	✓	✓	✓
orders	One row per customer order	id, user_id, status, payment_status, total_minor	✓	✓	✓	–
order_items	Order lines (name and price copied)	id, order_id, food_id, food_name, qty, unit_price_minor	✓	✓	–	–
notifications	Messages shown to users	id, user_id, title, message, is_read	✓	✓	✓	–
audit_logs	Reserved for an audit trail	id, actor_id, action, created_at (not written yet)	–	–	–	–

C = create, R = read, U = update, D = delete. Creation in *users* is registration, in *sessions* sign-in, in *notifications* system events; update in *orders* is the status change; delete in *foods* is deactivation (active = 0) and in *sessions* is sign-out. Relationships: users 1–N orders, orders 1–N order_items, categories 1–N foods, foods 1–N order_items and users N–N foods through carts. Foreign keys are enforced with PRAGMA foreign_keys=ON; CHECK constraints guard role, price, stock and quantity.

5. Graphical User Interface

The interface is a single-page Vue 3 application (`frontend/src/App.vue`). A top bar shows navigation that depends on the signed-in role. The style uses a green and white palette with the DM Sans and Outfit fonts, rounded cards, status pills and alert banners for errors and success messages.

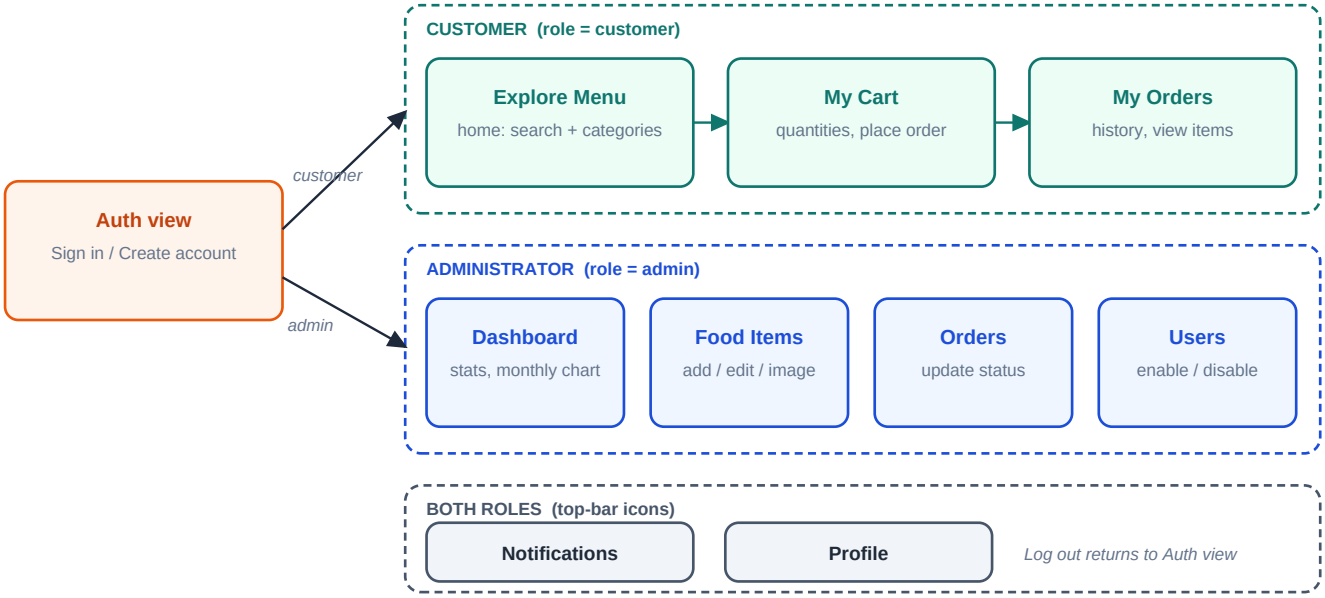


Figure 3 — Screen navigation flow by role

5.1 Views

View (page)	Role	Purpose and main controls
Sign in / Create account (auth)	Visitor	Tabs for sign-in (username or e-mail, password) and registration (name, username, e-mail, phone, account type; security code for administrators).
Explore Menu (home)	Customer	Hero banner, search box, category chips and food cards with price, stock count and an add button that is disabled when sold out.
My Cart (cart)	Customer	Quantity stepper, line totals, order summary and Place Order with a confirmation dialog.
My Orders (orders)	Customer	Order history with status, payment status and total; View items lists the order lines.
Notifications / Profile	Both	Unread highlighting and Mark all as read; edit full name and phone; change password.
Dashboard	Admin	Statistic cards (customers, food items, total orders, completed order value) and a monthly order bar chart.
Food Items (manage-food)	Admin	Add or edit form (category, description, price in LKR, stock, image upload) and the current menu with Edit and deactivate.
Orders (admin-orders)	Admin	All orders with customer, total, payment status and a status drop-down, locked once Completed or Cancelled.
Users	Admin	User list with role and Active/Disabled state; Enable or Disable button.

5.2 REST API summary (26 routes)

Area	Routes	Access
Account (6)	POST /api/auth/register, /api/auth/login, /api/auth/logout, /api/auth/password; GET /api/auth/me; PUT /api/profile	Public / signed in
Catalogue (3)	GET /api/health, /api/categories, /api/foods?q=	Public
Food admin (5)	POST /api/admin/foods; PUT and DELETE /api/admin/foods/<id>; POST /api/admin/foods/<id>/image; GET /uploads/<name>	Admin (images public)
Cart and orders (5)	GET /api/cart; PUT /api/cart/<id>; POST and GET /api/orders; GET /api/orders/<id>	Customer / owner
Notifications (2)	GET /api/notifications; POST /api/notifications/read	Signed in
Admin (5)	GET /api/admin/orders and /users and /analytics; PUT /api/admin/orders/<id>/status and /api/admin/users/<id>/active	Admin

6. Class List, Responsibilities and Class Diagram

Class / unit	Kind	Responsibility
St	struct (user-defined)	RAII wrapper around one sqlite3_stmt: the constructor prepares it, the destructor finalizes it, the overloaded b() binds text or integers, next() steps (throws on error), str() and num() read columns.
Identity	struct (user-defined)	Result of authentication: user id and role. An id of 0 means "not signed in".
crow::SimpleApp	library class	The application object: owns the routes and runs the HTTP server on port 8080 with four threads.
crow::request, crow::response, crow::json::wvalue (J)	library classes	Incoming request (headers, body, URL parameters), outgoing response and the JSON builder.
std::exception, std::runtime_error	library classes (inheritance)	runtime_error derives from exception. It is thrown for database and crypto failures and caught as const std::exception&.
std::mutex, std::lock_guard	library classes	Mutual exclusion; lock_guard releases the lock automatically when the handler ends.
Helper functions	file-scope functions	hex, randomhex, hashpw, sql, rows, reply, err, field, integer, token, who, admin, notice, init; plus main() with the 26 route handlers.
App (App.vue)	Vue component	Root component: reactive state (token, user, foods, cart, orders), computed values (isAdmin, filtered, cartTotal) and functions api(), run(), refresh(), checkout(), saveFood() and others.

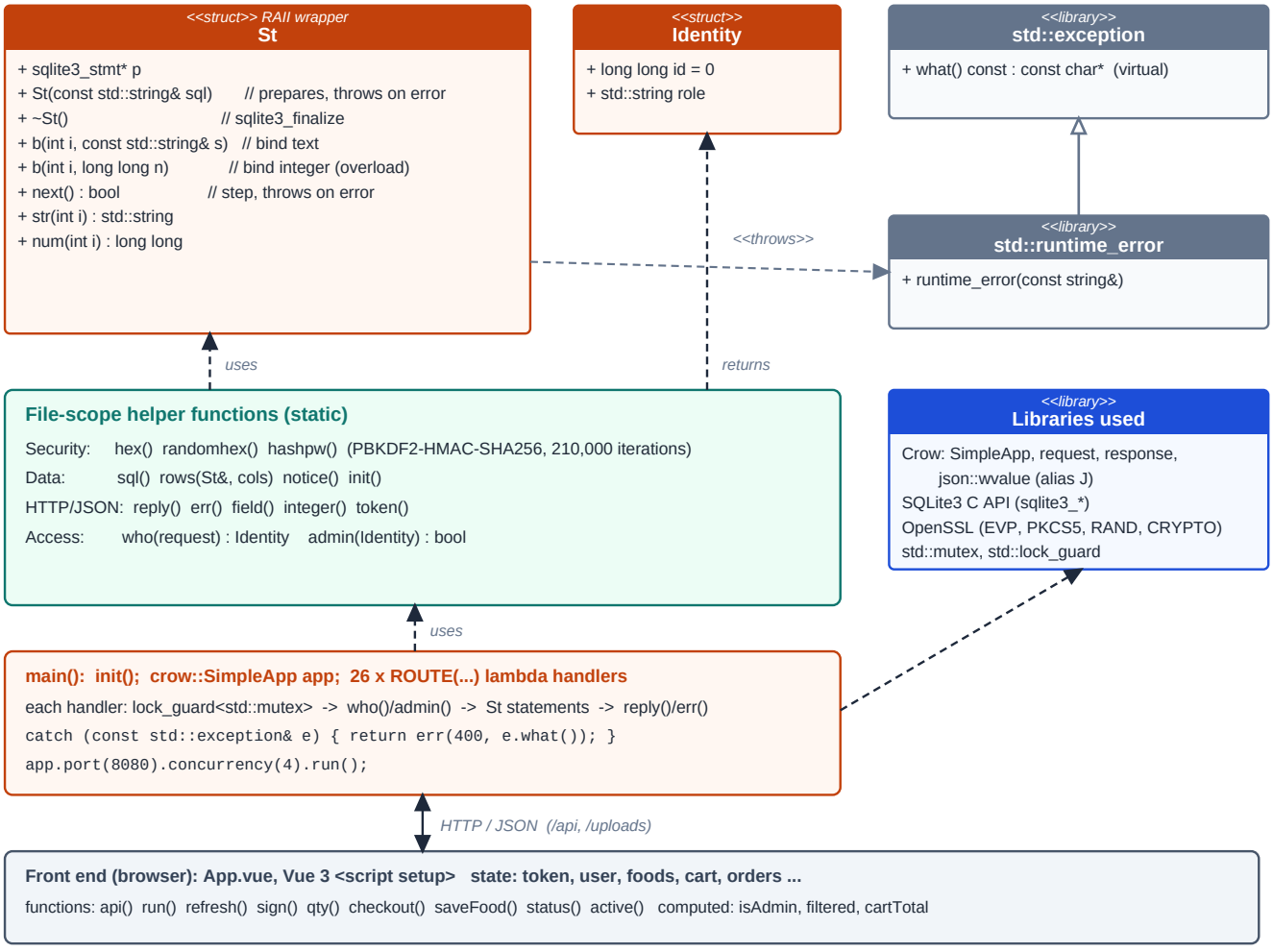


Figure 4 — Class and module diagram of Food Hub (hollow triangle = inheritance, dashed arrow = uses, + = public member)

7. OOP Concepts — Worked Examples from the Source Code

Each example below comes from the real files `backend/main.cpp` and `frontend/src/App.vue`. The project source is written in a compact style; the listings are reformatted over several lines for readability, without changing the code.

7.1 Class

A **class** is a blueprint that groups data and behaviour. `St` is the main user-defined class (declared with `struct`, so its members are public by default). It models one prepared SQL statement: the constructor acquires it, the destructor releases it, and the member functions bind parameters, step through rows and read columns.

```
// File: backend/main.cpp
struct St {
    sqlite3_stmt* p = nullptr;
    St(const std::string& s) {                              // constructor
        if (sqlite3_prepare_v2(db, s.c_str(), -1, &p, nullptr) != SQLITE_OK)
            throw std::runtime_error(sqlite3_errmsg(db));
    }
    ~St() { sqlite3_finalize(p); }                          // destructor
    void b(int i, const std::string& s) { sqlite3_bind_text(p, i, s.c_str(), -1, SQLITE_TRANSIENT); }
    void b(int i, long long n)          { sqlite3_bind_int64(p, i, n); }
    bool next() {
        int r = sqlite3_step(p);
        if (r != SQLITE_ROW && r != SQLITE_DONE) throw std::runtime_error(sqlite3_errmsg(db));
        return r == SQLITE_ROW;
    }
    std::string str(int i) { auto t = sqlite3_column_text(p, i); return t ? (const char*)t : ""; }
    long long num(int i)   { return sqlite3_column_int64(p, i); }
};
```

Listing 1 — The St class (backend/main.cpp)

7.2 Object

An **object** is a run-time instance of a class with its own state. Food Hub creates an `St` object for every statement a request needs; each holds its own `sqlite3_stmt*` and is destroyed when its scope ends. `Identity` objects carry the signed-in user, and the application itself is an object of the library class `crow::SimpleApp`.

```
// main(): the application object
crow::SimpleApp app;

// login handler: an St object with its own prepared statement and bound values
St s("SELECT id,full_name,role,salt,password_hash FROM users WHERE (username=? OR email=?) AND active=1");
s.b(1, field(j, "login"));
s.b(2, field(j, "login"));
if (!s.next()) return err(401, "Invalid credentials");

// who(): returns an Identity object (id 0 = not signed in)
struct Identity { long long id = 0; std::string role; };
if (s.next()) return { s.num(0), s.str(1) };
```

Listing 2 — Objects created from user-defined and library classes

7.3 Encapsulation

Encapsulation bundles data with the code that uses it and hides the internals. The database handle `db` and the mutex `mu` are `static` at file scope, so no other file can touch them. Raw SQLite calls (prepare, bind, step, finalize) are confined to `St`, and secrets are hidden behind functions: `hashpw()` is the only place that derives a password hash and `who()` the only place that turns a token into an identity. Because `St` is a struct its member `p` is still public; making it private is listed as an improvement.

```
static sqlite3* db = nullptr;      // internal linkage: hidden from other source files
static std::mutex mu;              // guards db; locked by every route handler

static std::string hashpw(const std::string& p, const std::string& salt) {
    ...
    if (!PKCS5_PBKDF2_HMAC(p.c_str(), (int)p.size(), s.data(), (int)s.size(),
                           210000, EVP_sha256(), 32, out))
        throw std::runtime_error("hash failed");
    return hex(out, 32);           // callers never see the algorithm details
}
static bool admin(Identity u) { return u.id && u.role == "admin"; }
```

Listing 3 — Hidden state and hidden algorithms

7. OOP Concepts (continued)

7.4 Abstraction

Abstraction exposes what an operation does and hides how it is done. A route handler never prepares statements, builds JSON by hand or formats HTTP headers: `St`, `rows()` and `reply()` hide that work. On the front end, `api()` hides fetch(), the Authorization header, JSON parsing and error handling behind one call.

```
// backend/main.cpp - a complete route: three calls hide prepare/step/finalize, JSON and HTTP headers
ROUTE("/api/categories")([] {
    std::lock_guard<std::mutex> lock(mu);
    St s("SELECT id,name FROM categories ORDER BY name");
    return reply(200, rows(s, 2));
});

// frontend/src/App.vue - one function hides fetch, headers, JSON and errors
async function api(path, method = 'GET', body = null, raw = false) { ... }
foods.value = await api('/foods');
```

Listing 4 — Abstraction through St, rows(), reply() and api()

7.5 Inheritance

Inheritance lets a class reuse and extend another class. The user-defined types `St` and `Identity` are standalone and do not derive from a base class, so the project's own code has no user-defined hierarchy yet; this is a known gap (Section 9). The code does use inheritance from the standard library: `std::runtime_error` derives from `std::exception`, so every thrown runtime_error is caught through its base class and has the inherited `what()` method.

```
// St constructor and sql(): throw a derived class
throw std::runtime_error(sqlite3_errmsg(db));

// route handlers that touch the database: catch through the BASE class reference
} catch (const std::exception& e) {
    return err(400, e.what());      // what() is inherited from std::exception
}
```

Listing 5 — Library inheritance used by the project (runtime_error is-a exception)

7.6 Polymorphism

Polymorphism means one interface with many behaviours. Food Hub uses it in three ways: (a) *overloading*, where the same name `b()` binds text or integers depending on the argument type; (b) *run-time polymorphism*, where the virtual `what()` called through a `std::exception&` runs the version of the actual thrown type; (c) *one registration interface for many handler shapes*, where `ROUTE` accepts lambdas with different parameter lists.

```
// (a) Overloading: same name, different parameter types
void b(int i, const std::string& s) { sqlite3_bind_text(p, i, s.c_str(), -1, SQLITE_TRANSIENT); }
void b(int i, long long n)          { sqlite3_bind_int64(p, i, n); }
s.b(1, field(j, "name"));              // text overload
s.b(4, integer(j, "price_minor"));     // integer overload

// (b) Run-time polymorphism: virtual what() called through the base-class reference
catch (const std::exception& e) { return err(400, e.what()); }

// (c) One ROUTE interface, three handler signatures
ROUTE("/api/health")([] { ... });
ROUTE("/api/auth/me")([](const crow::request& r) { ... });
ROUTE("/api/cart/<int>").methods(crow::HTTPMethod::PUT)([](const crow::request& r, int id) { ... });
```

Listing 6 — Overloading, run-time polymorphism and flexible handler signatures

7.7 Summary of the six concepts

Concept	Where it is shown	Evidence
Class	St, Identity	Members, constructor, destructor, methods
Object	St s(...), Identity, crow::SimpleApp app	New instances created at run time for each request
Encapsulation	static db and mu; St confines SQLite calls; hashpw(), who()	Hidden state and hidden algorithms
Abstraction	St, rows(), reply(), api()	Complexity hidden behind short calls
Inheritance	std::runtime_error derives from std::exception	Caught through the base class (library hierarchy; no user-defined hierarchy yet)
Polymorphism	Overloaded b(); virtual what(); ROUTE signatures	Same call, different behaviour

8. Validation and Business Rules

Area	Rule implemented in code
Registration	Name required, username 3+ characters, e-mail must contain @, password 8+ characters; username and e-mail are UNIQUE; an administrator needs the code from FOODHUB_ADMIN_CODE (403 otherwise).
Sign-in	Only active users; wrong credentials return the same 401 "Invalid credentials"; token is random, stored as a SHA-256 hash and expires after 24 hours.
Food items	Name required; price and stock cannot be negative (also CHECK constraints); only administrators; images PNG, JPEG or WebP up to 5 MB with signature check.
Cart	Customers only; quantity 0 to 100 (0 removes the line); a line is added only if the food is active and stock is sufficient.
Checkout	Customers only; fails if the cart is empty or any item is inactive or short of stock; total = sum of quantity x price; stock reduced, cart cleared, customer notified, all in one transaction.
Order status	Must be Pending, Confirmed, Preparing, Ready, Completed or Cancelled; a Completed or Cancelled order cannot change (409); the customer is notified.
Users and access	Administrators can enable or disable accounts but not their own; a customer sees only their own orders and order lines; passwords are changed only after the current one is verified.

9. Limitations and Future Improvements

- **Build and testing:** a full MSVC build has not been independently verified (see COMPILER_FIX.md) and there are no automated tests; a clean build and a test suite are the first next steps.
- **OOP structure:** the back end is one compact file. Introducing domain classes (User, Food, Order), an abstract Repository base class with derived implementations, and a private-member St would give real user-defined inheritance and polymorphism.
- **Assignment fit:** the brief asks for a desktop GUI (WinForms or MFC) and recommends MySQL; this web and SQLite version needs lecturer approval or a separate desktop client.
- **Missing features:** online payment, riders and vehicles, category management, e-mail, an administrator user-editing form and a written audit trail (audit_logs is created but unused). "Completed Order Value" is not verified revenue.
- **Hardening:** rate limiting, HTTPS, stricter order-status transitions and a review of session and upload handling are needed before any public deployment. The Chart.js dependency is not yet used.

10. Conclusion

Food Hub implements a complete small-business ordering workflow: secure sign-in, a searchable menu, a cart, transactional checkout with stock control, an administrator area for food items, orders and users, and a summary dashboard. The six OOP concepts are explained with real C++ code from the project; inheritance relies on the standard-library hierarchy, and the project differs from the brief by being a browser-based application on SQLite. These points are stated openly, and the improvements in Section 9 form a clear roadmap.

References and links. Crow, SQLite, OpenSSL, Vue 3 and Vite official documentation. Demo video: **https://youtu.be/PPedzbxg-5Q?si=rnJchHZDjowjFEqp** Source code: **https://github.com/chamilaki2004-ops/Food-delivery-software.git**